

05_m10_m12

M10 - Eval Harness (injector + scorer + 6-10 scenarios + naive baseline) [Wk2-v1 LEAN]

This is the v1 honest-eval deliverable and the trust centerpiece of the whole project. LEAN means: build `injector.py` + `scorer.py`, **reuse a 6-10 scenario subset** of the existing `eval/scenarios/scenarios.yaml` (do NOT regenerate the 50, do NOT build CI), and the naive "largest-absolute-delta" baseline. The eval runs **locally** (`python -m helios.eval`). The anti-overclaim move (the thing that wins interview rounds): make Claude state plainly that this proves *controlled attribution accuracy vs a baseline* and NOT real-world causal accuracy.

Context to load

- Auto-loaded: `CLAUDE.md` (canonical metrics, the core decomposition identity, the 85%-vs-45% targets, grounding rules G1-G5).
- The source of truth: `HELIOS_PROJECT_BIBLE.md` **sec 20** (Evaluation Framework) - specifically 20.2 injection mechanism (the two perturbation primitives), 20.4 metrics (top-1/top-3 segment accuracy, decomposition MAPE, dollar-at-risk error), 20.5 the naive baseline definition, 20.7 scoring details (normalized segment-key matching).
- `docs/LEAN_SCOPE.md` lines ~43, ~78, ~125 - the LEAN cut (6-10 scenarios, no CI) and the exact honesty sentence the eval must print.
- The format reference + reusable specs: `eval/scenarios/scenarios.yaml` (the 50-scenario master list) and the per-bucket files `eval/scenarios/01_single_segment_rate.yaml` ... `07_data_quality.yaml`. You are picking ~8 of these, not authoring new ones.
- The thing being graded: the M7 grounded LLM brief output (the JSON diagnosis your minimal loop emits) and `stats-mcp.decompose_change` (M6b) - the scorer reads their structured output.
- /clear before starting (D8). M10 is its own focused session.

Files to provide

- `@HELIOS_PROJECT_BIBLE.md` (sec 20 only - paste/attach the section, not the whole Bible).
- `@eval/scenarios/scenarios.yaml` (the master format reference) and the picked subset files.
- `@models/semantic/semantic_layer.yaml` and the GA4 schema - the scorer's `hallucination` check needs the canonical column/metric registry (D7).

- The M7 diagnosis output schema (the dict your loop returns: predicted `root_cause_segment`, predicted `dominant_effect`, the decomposition `{mix, rate, interaction}`, `revenue_at_risk_usd`).
- `@helios/mcp/stats.py` (the `decompose_change` contract the harness drives) and `@helios/mcp/warehouse.py` (`run_query` / `dry_run` to materialize `helios_eval.fct_daily_funnel_perturbed`).

Prompts to use

Pick the subset first (cheap, plan-mode, no codegen):

```
[ROLE] You are setting up the Helios LEAN eval (milestone M10). Plan only - no code yet.
[SOURCE OF TRUTH] Bible sec 20.3 (coverage buckets) + the attached eval/scenarios/scenarios.yaml.
[TASK] From the existing 50 scenarios, SELECT exactly 8 to form the LEAN benchmark, covering:
  2 single_segment_rate (e.g. S001, S002), 2 single_segment_mix (e.g. S011, S014 the Simpson's adver
  1 multi_segment_rate (e.g. S021), 1 multi_segment_mixed (e.g. S027), 2 no_anomaly_control (e.g. S0
[CONSTRAINTS] Do NOT write new scenarios and do NOT modify the YAML. Reuse the exact specs as-is.
  Justify each pick against the bucket it covers and why it discriminates Helios from the naive base
  (the mix + adversarial picks are the ones the baseline must fail).
[OUTPUT] A table: scenario_id, bucket, why-picked. Cite the bucket rows in Bible sec 20.3.
```

Then the injector (TDD, one file):

```
[ROLE] You are implementing Helios milestone M10, file helios/eval/injector.py.
[SOURCE OF TRUTH] Copy/adapt the injection mechanism in Bible sec 20.2 (attached). Follow CLAUDE.md.
[UPSTREAM] Read scenario specs from eval/scenarios/*.yaml (attached format). Materialize via
  warehouse-mcp.run_query/dry_run (helios/mcp/warehouse.py). Operate ONLY on helios_eval - never the
[TASK] Implement inject(spec) with the TWO primitives from sec 20.2 and NOTHING else:
  - rate perturbation: recompute numerator = round(sessions * base_rate * rate_multiplier), hold ses
  - volume/mix perturbation: multiply segment sessions by volume_multiplier, hold per-step rates fix
  Write helios_eval.fct_daily_funnel_perturbed + a labels record (scenario_id, anomaly_type, affecte
[CONSTRAINTS] Use ONLY canonical column names (sessions, view_item_sessions, add_to_cart_sessions,
  begin_checkout_sessions, purchasing_sessions, revenue, transactions) and the fct_daily_funnel grai
  invent columns. If a name is not in semantic_layer.yaml / the GA4 schema, STOP and ASK.
[TEST-FIRST] Write tests/eval/test_injector.py FIRST: for a rate scenario assert sessions are
  byte-identical pre/post and only the numerator changed; for a mix scenario assert each segment's p
  rate is unchanged and total sessions is conserved within tolerance. Run `pytest tests/eval/test_in
  and show output.
[OUTPUT] Cite the sec 20.2 sub-bullet used for each primitive.
```

Then the scorer + naive baseline (TDD, one file each - the honesty lever lives here):

```
[ROLE] You are implementing Helios milestone M10, files helios/eval/scorer.py and helios/eval/baseli
[SOURCE OF TRUTH] Bible sec 20.4 (metrics) + 20.5 (naive baseline) + 20.7 (scoring details). Follow
[UPSTREAM] Read predicted diagnosis from the M7 loop output dict and ground-truth from helios_eval.l
```

[TASK]

baseline.py: implement the naive "largest-absolute-segment-delta" analyst exactly per sec 20.5 - for the anomalous metric, $\Delta_i = \text{value_at}(t1) - \text{value_at}(t0)$ per segment, rank by $|\Delta_i|$, de top segment the root cause. NO mix-vs-rate decomposition (that is the point - it gets fooled by scorer.py: compute, in DETERMINISTIC PYTHON ONLY (NEVER an LLM judge), per sec 20.4: top-1 and top-3 root-cause segment accuracy (normalized sorted dimension=value key match, sec 20 decomposition MAPE on {mix, rate, interaction} (controls excluded), dollar-at-risk abs % error, and an AST-based hallucination check against semantic_layer.yaml + GA4 schema.

[CONSTRAINTS] The scorer must be pure Python/scipy - do NOT call Claude to grade. Use only canonical HONESTY: the final report MUST print a one-line caveat verbatim: "This benchmark measures controll attribution accuracy (where the number moved) against a naive baseline. It does NOT prove real-wor accuracy - the cause (a deploy, a price change) lives outside this frozen dataset."

[TEST-FIRST] Write tests/eval/test_scorer.py FIRST with a hand-worked golden case (a known mix scena the baseline picks the wrong high-volume segment and the decompose-based prediction picks the righ assert: baseline top-1 = miss, Helios top-1 = hit, MAPE within tolerance. Run pytest and show outp

[OUTPUT] Cite sec 20.4/20.5/20.7 per function.

Preventing hallucinations

- D2/D3 specialized: the scenarios already exist with exact ground-truth labels - hand Claude scenarios.yaml and forbid it from writing new specs. The number-one drift here is Claude "improving" the scenarios; the prompt says reuse as-is.
- D6 (let the machine catch it): the injector is graded by its own invariants - the test_injector.py rate-test asserts sessions are byte-identical pre/post (a rate perturbation that touches volume is a bug); the mix-test asserts per-segment rates are unchanged. These are the silent-failure tripwires from CLAUDE.md sec 8.
- D7 hard rule: the scorer's hallucination check is itself the column-name guard - point it at semantic_layer.yaml + the GA4 schema; any emitted column not in those is a fail. Keep both attached.
- The honesty/anti-overclaim guard is a *generation* constraint, not just review: the prompt forces the verbatim caveat line and forbids claiming "causal" or ">=85% real-world accuracy." If Claude writes "Helios proves it diagnoses *why*," reject - the doc (LEAN_SCOPE line ~125) only permits "*where* it moved and *how much it's worth*."
- D5: if Claude reaches for a scoring metric not in sec 20.4 (e.g. an "LLM faithfulness judge"), STOP - faithfulness/Critic-as-judge is [v2], explicitly cut from LEAN. The LEAN scorer is four deterministic numbers, period.

Reviewing generated code

- R1: each scorer function cites sec 20.4/20.5/20.7; the injector cites the two 20.2 primitives.
- R3 (run it): `python -m helios.eval` end-to-end on the 8 scenarios. **The two sanity checks that matter most** (this is the review, not a formality):
 1. The baseline beats *nothing* - it must score clearly above 0 on the pure single-segment-rate scenarios (S001/S002) and clearly *fail* the mix/adversarial ones

(S011/S014). If the baseline scores ~0 everywhere your baseline is broken (too weak a strawman); if it scores high on mix, it isn't the largest-delta rule.

2. Helios beats the baseline - top-1 materially higher than the baseline, driven by the mix scenarios the decomposition handles. If they tie, the eval proves nothing - investigate before reporting a number.

- R4/R6: grep the scorer + injector emitted SQL/columns against `semantic_layer.yaml` ; hallucination count must be 0. Confirm the scorer is **not** graded by an LLM (read `scorer.py` - it must be scipy/Python, no `anthropic / messages.create` import). This is a hard reject if violated.
- R5: read the report output - it must print the honesty caveat verbatim and must NOT print an unqualified "85% causal accuracy." Report the *actual measured* numbers on your 8 scenarios, not the Bible's aspirational 0.882.
- Budget note: do NOT re-run the M7 LLM brief on every scenario during dev (that burns Pro usage). Develop injector + scorer + baseline against *recorded* M7 outputs / fixtures; run the live LLM loop through the 8 scenarios ONCE to validate v1.
- Per-scenario debugging: have the harness emit a per-scenario JSON (predicted vs label, every sub-score) per Bible sec 20.6 so a miss is inspectable. When you find a miss, read that JSON before re-prompting Claude - usually it is a segment-key normalization bug (sec 20.7: sorted dimension=value match), not a logic error, and you fix the matcher, not the diagnosis.

Slash command to create here (reuse later). `/new-scenario` - scaffold one eval scenario (the full spec block + its expected ground-truth labels) from a one-line description, validated against the `scenarios.yaml` field set in CLAUDE.md. Keep it for when v2 reintroduces buckets (seasonality-decoy, data-quality) the LEAN 8 omitted; for v1 you mostly *select* existing scenarios rather than author new ones, so this command stays dormant until you genuinely need an uncovered bucket. Pair it with a quick re-run of the injector invariants test so a hand-written spec cannot silently violate the rate/mix primitives.

M11 - Autonomy & depth [v2-later]

Out of the 2-week / Claude-Pro scope, and partly theater on a frozen dataset. The red-team explicitly killed "always-on / scheduler / autonomous" (LEAN_SCOPE line ~39: "Run it as a command. Why: theater on a frozen dataset."). A scheduler that re-diagnoses a static 2020-11-01->2021-01-31 export produces the same answer every run - the autonomy is real engineering but adds no signal on this data. Forecasting/cohort depth (Bible sec 23.3) is genuine v2, deferred for time, not because it is unsound.

How you would drive Claude Code later (sketch). /clear, then plan-mode against Bible sec 23.3 + the deferred-items table (sec 23.7). Build in three thin slices, each its own session: (1) a scheduler wrapper - Claude scaffolds a Cloud Scheduler/cron entry that just invokes the existing `python -m helios.run`; no new diagnosis logic. (2) Forecast wiring - `[ROLE] implement stats-mcp.forecast (prophet/pmdarima)` per Bible sec 23.3; `expected-vs-actual residual feeds Monitor.detect_anomaly`; TDD with a golden series so the forecast is deterministic-seeded math, not prose (G3). (3) `cohort_retention / rfm_segment` feeding the Diagnose hypothesis tree - but cap retention at the 30/60-day proxy the data supports (sec 23.7: the ~3-month window is too short for true LTV; have Claude assert and print that limit rather than fabricate a curve). The Critic refutation battery (mix-shift/sample/seasonality/data-quality) also lands here.

Anti-hallucination + honesty (the v2 carry-over). Same doctrine: source-of-truth section attached, only canonical names, every stat through `stats-mcp` seeded. The specific overclaim to police: a scheduler does not make Helios "autonomous" on frozen data, and forecast residuals on a closed historical export are a demo, not a live signal - Claude must label them as such. Review by running the forecast against a held-back window and checking residual sign/magnitude by hand; never accept a forecast Claude narrated without the tool output.

M12 - Productionization & frontier [v2-later]

Furthest out of scope; the frontier half is partly theater on this dataset by construction.

Bible sec 23.4 (multi-tenant, real-time streaming, warehouse-agnostic) and 23.5 (true causal inference, closed-loop experimentation) are weeks of platform + causal-ML work for one dev. More importantly, the red-team's central honesty point (LEAN_SCOPE line ~125): the public data has **no experiment assignment** (sec 23.7), so true causal inference cannot be *validated* here - the frontier exit criterion ("a causal estimate validated against a held-out randomized experiment") is unmeetable on this sample. Multi-tenant and streaming add no signal to a single frozen export. These earn their place on the *roadmap narrative*, not the build.

How you would drive Claude Code later (sketch). /clear; plan against sec 23.4-23.5 + the capability-maturity table (sec 23.6). Slices, each isolated: (1) warehouse-agnostic adapters behind `warehouse-mcp` (Snowflake/Databricks/DuckDB) - `[ROLE] implement a DuckDB adapter behind the existing warehouse-mcp contract`; review with the exit criterion "two warehouses pass identical `reconcile` tests" (sec 23.4) - the semantic layer stays the only SQL author regardless of dialect. (2) Tenant isolation - per-tenant semantic layer + byte budgets; (3) causal methods (diff-in-diff / synthetic control / double-ML) *only* where data supports it, replacing

correlational decomposition - and Claude must gate each on a data-sufficiency check, refusing to emit a causal estimate the sample cannot support.

Anti-hallucination + honesty. This is where overclaiming is most dangerous and most tempting. The hard rule for Claude: never let the Narrator upgrade "decomposition attributes *where* the change is" into "Helios proves *why*" - the cause lives outside the data (LEAN_SCOPE line ~125, the maturity move). Any causal language must carry the confound caveat the Critic attaches. Review by demanding the data-sufficiency assertion in code (a causal estimate without an assignment mechanism is rejected) and by reading every frontier brief for unqualified causal claims - on a frozen observational sample those are the exact statements that "get you destroyed in an interview" (LEAN_SCOPE line ~14). For v2, the deliverable is honest roadmap prose plus the scoping rationale, not running causal code against data that cannot validate it.